

Ostacoli per un lama

Un lama vuole attraversare l'altopiano andino, e ha una mappa dell'altopiano come griglia di $N \times M$ celle quadrate. Le righe della mappa sono numerate da 0 a $N - 1$ dall'alto verso il basso, mentre le colonne sono numerate da 0 a $M - 1$ da sinistra a destra. La cella della mappa nella riga i e nella colonna j ($0 \leq i < N, 0 \leq j < M$) è indicata con (i, j) .

Il lama ha studiato il clima dell'altopiano e ha scoperto che tutte le celle di ogni riga della mappa hanno la stessa **temperatura** e tutte le celle di ogni colonna della mappa hanno la stessa **umidità**. Il lama ha quindi ottenuto due array di interi T e H di lunghezza N e M rispettivamente, dove $T[i]$ ($0 \leq i < N$) indica la temperatura delle celle nella riga i , e $H[j]$ ($0 \leq j < M$) indica l'umidità delle celle nella colonna j .

Il lama ha poi studiato anche la flora e ha notato che una cella (i, j) è **priva di vegetazione** se e solo se la sua temperatura è maggiore della sua umidità, formalmente $T[i] > H[j]$.

Il lama può attraversare l'altopiano solo seguendo **percorsi validi**. Un percorso valido è una sequenza di celle distinte che soddisfano le seguenti condizioni:

- Ogni coppia di celle consecutive nel percorso condivide un lato comune.
- Tutte le celle del percorso sono prive di vegetazione.

Il vostro compito è rispondere a Q domande, ciascuna identificata da quattro numeri interi L, R, S , e D , in cui dovete determinare se esiste un percorso valido tale che:

- Il percorso inizia nella cella $(0, S)$ e termina nella cella $(0, D)$ (è garantito che entrambe le celle siano prive di vegetazione).
- Tutte le celle del percorso si trovano nelle colonne da L a R comprese.

Dettagli di implementazione

La prima funzione da implementare è:

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : un array di lunghezza N che specifica la temperatura in ogni riga.
- H : un array di lunghezza M che specifica l'umidità in ogni colonna.
- Questa funzione viene chiamata esattamente una volta per ogni caso di test, prima di qualsiasi chiamata a `can_reach`.

La seconda funzione da implementare è:

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : numeri interi che descrivono una domanda.
- Questa funzione viene chiamata Q volte per ogni caso di test.

Questa funzione deve restituire `true` se e solo se esiste un percorso valido dalla cella $(0, S)$ alla cella $(0, D)$, e tutte le celle del percorso si trovano nelle colonne da L a R , comprese.

Assunzioni

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$ per ogni i tale che $0 \leq i < N$.
- $0 \leq H[j] \leq 10^9$ per ogni j tale che $0 \leq j < M$.
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- Entrambe le celle $(0, S)$ e $(0, D)$ sono prive di vegetazione.

Subtask

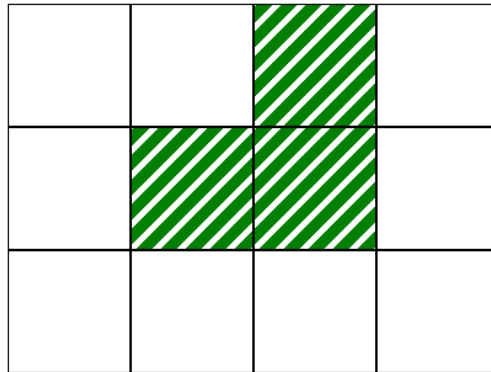
Subtask	Punteggio	Assunzioni aggiuntive
1	10	$L = 0, R = M - 1$ per ogni domanda. $N = 1$.
2	14	$L = 0, R = M - 1$ per ogni domanda. $T[i - 1] \leq T[i]$ per ogni i tale che $1 \leq i < N$.
3	13	$L = 0, R = M - 1$ per ogni domanda. $N = 3$ e $T = [2, 1, 3]$.
4	21	$L = 0, R = M - 1$ per ogni domanda. $Q \leq 10$.
5	25	$L = 0, R = M - 1$ per ogni domanda.
6	17	Nessun vincolo aggiuntivo.

Esempio

Si consideri la seguente chiamata:

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

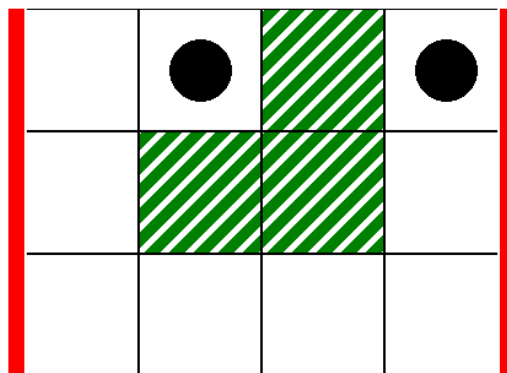
Ciò corrisponde alla mappa dell'immagine seguente, dove le celle bianche sono prive di vegetazione:



Come prima domanda, si consideri la seguente chiamata:

```
can_reach(0, 3, 1, 3)
```

Ciò corrisponde allo scenario dell'immagine seguente, dove le linee verticali spesse indicano l'intervallo di colonne da $L = 0$ a $R = 3$, e i dischi neri indicano le celle iniziali e finali:



In questo caso, il lama può arrivare dalla cella $(0, 1)$ alla cella $(0, 3)$ attraverso il seguente percorso valido:

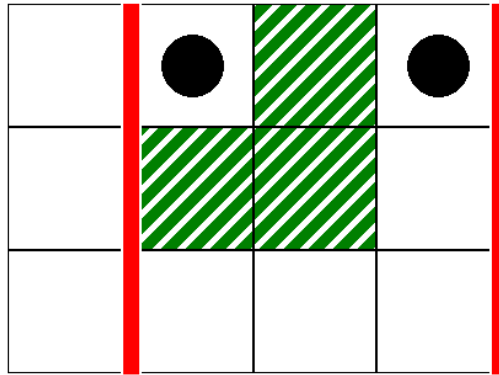
$(0, 1), (0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (1, 3), (0, 3)$

Pertanto, questa chiamata dovrebbe restituire `true`.

Come seconda domanda, si consideri la seguente chiamata:

```
can_reach(1, 3, 1, 3)
```

Questo corrisponde allo scenario dell'immagine seguente:



In questo caso, non esiste un percorso valido dalla cella $(0, 1)$ alla cella $(0, 3)$, tale che tutte le celle del percorso si trovino all'interno delle colonne da 1 a 3, comprese. Pertanto, questa chiamata dovrebbe restituire `false`.

Grader di esempio

Formato di input:

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

Dove $L[k], R[k], S[k]$ e $D[k]$ ($0 \leq k < Q$) specificano i parametri per ogni chiamata a `can_reach`.

Formato di output:

```
A[0]
A[1]
...
A[Q-1]
```

In questo caso, $A[k]$ ($0 \leq k < Q$) è 1 se la chiamata `can_reach(L[k], R[k], S[k], D[k])` ha restituito `true`, e 0 altrimenti.